



Ca' Foscari
University
of Venice

**Department
of Economics**

Working Paper

**Francesco Bertoluzzo and
Marco Corazza**

**Reinforcement Learning
for automatic financial trading:
Introduction and some applications**

ISSN: 1827/3580
No. 33/WP/2012



Reinforcement Learning for automatic financial trading: Introduction and some applications

Francesco Bertoluzzo

(fbertoluzzo@unive.it)

*Department of Economics
Ca' Foscari University of Venice*

Marco Corazza

(corazza@unive.it)

*Department of Economics
Ca' Foscari University of Venice
and
Advanced School of Economics of Venice*

First Draft: December 3, 2012

Abstract

The construction of automatic Financial Trading Systems (FTSs) is a subject of research of high interest for both academic environment and financial one due to the potential promises by self-learning methodologies and by the increasing power of actual computers. In this paper we consider Reinforcement Learning (RL) type algorithms, that is algorithms that optimize their behavior in relation to the responses they get from the environment in which they operate, without the need for a supervisor. In particular, first we introduce the essential aspects of RL which are of interest for our purposes, then we present some original automatic FTSs based on differently configured RL algorithms and apply such FTSs to artificial and real time series of daily financial asset prices.

Keywords

Financial Trading System; Reinforcement Learning; Stochastic control; Q-learning algorithm; Kernel-based Reinforcement Learning.

JEL Codes

C61; C63; D83; G11.

Address for correspondence:

Marco Corazza

Department of Economics
Ca' Foscari University of Venice
Cannaregio 873, Fondamenta S.Giobbe
30121 Venezia – Italy
Phone: (+39) 041 2346921
Fax: (+39) 041 2349176
E-mail: corazza@unive.it

This Working Paper is published under the auspices of the Department of Economics of the Ca' Foscari University of Venice. Opinions expressed herein are those of the authors and not those of the Department. The Working Paper series is designed to divulge preliminary or incomplete work, circulated to favour discussion and comments. Citation of this paper should consider its provisional character.

1 Introduction

Many resources have been recently spent among professionals as well as in the academic world in searching for automatic *Financial Trading Systems* (FTSs) that can substitute for experience, capacity, and intuition human operators in the choice of investments in financial markets.

In the specialized literature several different approaches have been considered to address such a problem: Statistical and econometric models using, among others, macroeconomic indicators; Simpler and flexible models based on the Technical Analysis indicators; Models based on machine learning that make extensive use of computing capacity.

In this paper we consider a class of self-adaptive algorithms for dealing with the above problem in order to obtain profitable and fully automatic FTSs. In particular, the methodology we consider is known as *Reinforcement Learning* (RL) [1] also known as *Neuro-Dynamic Programming* [2]. In our approach, investment decision making is viewed as a stochastic control problem and the investment strategies are discovered directly interacting with the market. So, the need to build forecasting models for future prices or returns is eliminated.

In literature there are many contributions in this research field. Among the ones we recall [6], [7], and [5]. In general, they show that such strategies perform better than the ones based on supervised learning methodologies when market frictions are considered. In [6] a simplified version of RL algorithm, called Direct Learning, is used in order to set a FTS that, taking into account transaction costs, maximizes an appropriate investor's utility function based on a differential version of the well-known Sharpe ratio. Then, they show by controlled experiments that the proposed FTS performs better than standard FTSs. Finally, the authors use their FTS to make profitable trades with respect to assets of the U.S. financial markets. In [7], the authors mainly compare FTSs developed by using RL methodologies with FTSs developed by using stochastic dynamic programming methodologies. In general they show by extensive experiments that the former approach is better than the latter one. In [5] the author considers an FTS similar to the one developed in [6] and applies it to the financial high-frequency data, obtaining profitable performances.

With respect to the prominent literature, in this paper we will do the following:

- We will develop and we will apply FTSs based on two different RL approaches, namely the *Temporal Difference* one (see subsection 2.3) and the *Kernel-based Reinforcement Learning* (see subsection 2.4);
- Beyond to consider the usual buy and sell signals, we will take into account also a third signal: The stay-out-from-the-market one;
- Instead of using the differential Sharpe ratio as performance indicator, we will utilize the classical Sharpe ratio computed on the last $L \in \mathbb{N}$ trading days.

The remainder of the paper is organized as follows: In section 2 we will introduce the essential aspects of RL which are of interest for our purposes; In section 3 we will present our RL-based FTSs and we will provide the results of their applications to artificial and real time series; In section 4 we will give some concluding remarks.

2 Reinforcement Learning

Learning by interacting directly with the environment without the need of a supervisor is likely the more immediate idea about the nature of learning. The consequences of the actions of the learner (agent) lead the agent himself to choose what are the actions that allow to obtain the desired results and what are the ones to avoid. RL formalizes this kind of learning by maximizing a numerical reward ([1]). The agent has to discover which actions yield the most reward by trying them. RL is different from Supervised Learning (SL) in which the agent learns from examples provided by an external supervisor. SL is an important kind of learning, but in interactive problems it is often impractical to obtain examples of desired behavior that are representative of the situation in which the agent has to act. In uncharted and unknown environments (like, for instance, financial markets) the agent must be able to learn only from its own past experience.

To formalize these first ideas, let us consider a system observed at discrete time steps in which the state at time step t , $s_t \in \mathcal{S}$, summarizes all information concerning the system available to the agent. In the RL framework it is assumed that the system satisfies the Markov property, that is that the probability of transition from the actual state s_t to the next one s_{t+1} depends only on the current state s_t . On the basis of s_t , the agent selects an action $a_t \in \mathcal{A}(s_t)$, where $\mathcal{A}(s_t)$ is the set of all possible actions the agent can take given the state s_t . At time step $t + 1$ the agent receives a reward, $r(s_t, a_t, s_{t+1}) \in \mathbb{R}$, as consequence of his actions a_t and of the new state s_{t+1} in which he finds himself. The reward is a numerical representation of the satisfaction of the agent. Generally, the agent wish to maximize the expected value of some global return, $R(s_t)$, which is defined as function of the actual reward and of the future discounted ones. Without specifying the chosen action as argument, this function can be written as:

$$R(s_t) = r(s_t, a_t, s_{t+1}) + \gamma r(s_{t+1}, a_{t+1}, s_{t+2}) + \gamma^2 r(s_{t+2}, a_{t+2}, s_{t+3}) + \dots,$$

where $\gamma \in (0, 1)$ is the discount factor.

In RL a policy $\pi(s_t) = a_t$ is a mapping from states to actions defining the choice of action a_t given state s_t . In order to maximize the expected $R(s_t)$, RL searches for a suitable policy. Considering also the policy $\pi(\cdot)$ we can write the global return as:

$$R^\pi(s_t) = r(s_t, \pi(s_t), s_{t+1}) + \gamma r(s_{t+1}, \pi(s_{t+1}), s_{t+2}) + \gamma^2 r(s_{t+2}, \pi(s_{t+2}), s_{t+3}) + \dots$$

2.1 Value functions

RL approaches are generally based on estimating *value functions*. These functions of states (or state-action pairs) attribute a value to each state s_t (or state-action pairs) proportional to the rewards achievable in the future from the current state s_t (or state-action pairs). They evaluate how good is for the agent to be in a given state (or to perform a given action in a given state). The notion “how good” is defined in terms of R_t . In particular, the value of a state $s_t = s$ following policy $\pi(\cdot)$ is the expected sum of the current and the future discounted rewards when starting in state $s_t = s$ and thereafter following policy $\pi(\cdot)$, that is:

$$V^\pi(s) = E[R^\pi(s_t) | s_t = s].$$

Similarly, the value of taking action $a_t = a$ being in state $s_t = s$ under policy $\pi(\cdot)$ is the expected sum of the current and the future discounted rewards starting from state $s_t = s$, taking action $a_t = a$ and thereafter following policy $\pi(\cdot)$, that is:

$$Q^\pi(s, a) = E[R^\pi(s_t) | s_t = s, a_t = a].$$

A fundamental property of value functions is that they satisfy particular recursive relationships. For any policy $\pi(\cdot)$ and any state $s_t = s$, the following consistency condition holds between the value of s_t and the value of any possible successor state s_{t+1} :

$$V^\pi(s) = E[r(s_t, \pi(s_t), s_{t+1}) + \gamma V^\pi(s_{t+1}) | s_t = s]. \quad (1)$$

Equation (1) is the *Bellman equation* for $V^\pi(s_t)$. One can prove that the value $V^\pi(s)$ is the unique solution to its Bellman equation.

2.2 Generalized policy iteration

Task of RL consists in finding an optimal policy, that is a policy which is better than or equal to all the other policies. The optimal policy identifies the values $V^*(s)$ and $Q^*(s, a)$ such that

$$V^*(s) = \max_{\pi} V^\pi(s) \text{ and } Q^*(s, a) = \max_{\pi} Q^\pi(s, a), \quad (2)$$

for all $s \in \mathcal{S}$ and for all $a \in \mathcal{A}(s)$. Since $V^*(s)$ is the value function for a policy, it satisfies the Bellman equation (1). Because it is also the optimal value function, $V^*(s)$'s Bellman condition can be written in a special form without reference to any specific policy. This form is the Bellman equation for $V^*(s)$, or the *Bellman optimality equation*, which expresses the fact that the value of a state under an optimal policy must equal the expected global return for the best action from the state itself, that is:

$$V^*(s) = \max_a Q^*(s, a) = E[R^*(s_t) | s_t = s, a_t = a]. \quad (3)$$

With equivalent arguments, the Bellman optimality equation for $Q^*(s, a)$ is:

$$Q^*(s, a) = E \left[r(s_t, a_t, s_{t+1}) + \gamma \max_{a'} Q^*(s_{t+1}, a') | s_t = s, a_t = a \right].$$

At this point, it is possible to iteratively calculate the value function for a state (or for a state-action pair). Let $V_0^\pi(s_t)$ for all $s_t = s \in \mathcal{S}$ be an arbitrarily initialization of the state value function. Each successive approximation is obtained by using the Bellman equation as an update rule:

$$\hat{V}_{k+1}^\pi(s_t) = E \left[r(s_t, a_t, s_{t+1}) + \gamma \hat{V}_k^\pi(s_{t+1}) \right] \quad (4)$$

for all $s_t = s \in \mathcal{S}$. If the expectation $\hat{V}_{k+1}^\pi(s_t)$ exists, then $\lim_{k \rightarrow +\infty} \hat{V}_k^\pi(s) = V^\pi(s)$.

The reason for computing the value functions for a policy is to find better policies in order to increase the expected value of the global returns. This process is called *policy improvement*.

As described, the policy improvement process requires the evaluation of the previous policy. This evaluation can be made by (4), which is itself an iterative process that converges in the limit.

Fortunately, there is no need to wait for the exact convergence, in fact one can stop the policy evaluation iteration in several ways without losing the convergence ([1]). An important special case is when policy evaluation is stopped after just one step. One can combine the evaluation process and the improvement one by stopping the policy evaluation process at each step and then by improving the policy itself. This mixed algorithm is called *generalized policy iteration*. It can be written as:

$$\widehat{V}_{k+1}^{\pi}(s_t) = \max_a \mathbb{E} \left[r(s_t, a, s_{t+1}) + \gamma \widehat{V}_k^{\pi}(s_{t+1}) \right], \quad (5)$$

where $\widehat{V}_{k+1}^{\pi}(s_t)$ is the update estimate with the improved policy at step $k+1$ with respect to the old estimate and the old policy at step k .

In order to improve the policy we chosen an approach, among the ones presented in literature, which may produce increasing of the global return in the long run. Following such an approach, the choice of the action at each time step t is given by:

$$a_t = \begin{cases} \pi'(s_t) & \text{with probability } 1 - \varepsilon \\ a \in \mathcal{A}(s_t) & \text{with probability } \varepsilon, \end{cases}$$

where $\varepsilon \in (0,1)$ and $\pi'(s_t)$ is the candidate action which maximizes $Q^{\pi}(s, a)$.

In the next subsections we will introduce the two different methods we will use in our FTSs for calculating the expected value (5). Note that we can not take into account methods like the *Dynamic Programming*-based ones and the *Monte Carlo*-based ones. In fact:

- The former needs a model to calculate the real probabilities of transition from a state to another one, whereas in the financial trading such a model is generally not known or not available;
- The latter, in order to improve the policy, needs to wait for until the end of all the trades, whereas a FTS trades an indefinite number of times.

2.3 Temporal Difference methods and Q-Learning algorithm

In this subsection we present a class of policy evaluation algorithms known as *Temporal Difference methods* (TDms) which update step by step the estimate of $V(s_t)$. First of all one puts in evidence that it is possible to write $\widehat{V}_{k+1}(s_t)$ in the following recursive way:

$$\widehat{V}_{k+1}(s_t) = \frac{1}{k+1} \sum_{j=1}^{k+1} R_j(s_t) = \frac{1}{k+1} \left[R_{k+1}(s_t) + \sum_{j=1}^k R_j(s_t) \right] = \dots = \widehat{V}_k(s_t) + \alpha_k \left[R_{k+1}(s_t) - \widehat{V}_k(s_t) \right],$$

where $\alpha_k = 1/(k+1)$. The TDms can update the estimate $\widehat{V}_{k+1}(s_t)$ as soon as the quantity

$$d_k = R_{k+1}(s_t) - \widehat{V}_k(s_t) = r(s_t, s_{t+1}) + \gamma \widehat{V}_k(s_{t+1}) - \widehat{V}_k(s_t),$$

becomes available. Therefore the above recursive relationship can be rewritten as:

$$\widehat{V}_{k+1}(s_t) = \widehat{V}_k(s_t) + \alpha_k \left[r(s_t, s_{t+1}) + \gamma \widehat{V}_k(s_{t+1}) - \widehat{V}_k(s_t) \right].$$

With regard to the convergence of the TDms, one can prove that, if $\sum_{k=1}^{+\infty} \alpha_k = +\infty$ and $\sum_{k=1}^{+\infty} \alpha_k^2 < +\infty$, then $\lim_{k \rightarrow +\infty} \Pr \left\{ \left| \widehat{V}_k(s_t) - V(s_t) \right| < \varepsilon \right\} = 1$, for any $\varepsilon > 0$ ([2]).

The TDms are “naturally” developed in an incremental and on-line fashion which make them particularly appealing for the building of FTSs. In literature there exist several different TDms, the most widespread of whom is the *Q*-Learning algorithm (QLa). The QLa is an *off*-policy control method, where off indicates that two different policies are used in the policy improvement process: A first one is used to estimate the value functions, another is used to control the improvement process. One can prove that the so-obtained state-action value function is given by:

$$\widehat{Q}_{k+1}(s_t, a_t) = \widehat{Q}_k(s_t, a_t) + \alpha_k \left[r_{t+1} + \gamma \max_a \widehat{Q}_k(s_{t+1}, a_{t+1}) - \widehat{Q}_k(s_t, a_t) \right]. \quad (6)$$

Up to now we have assumed that the states are discrete variables which, even more, assume a limited number of values. But generally a system like a financial market is characterized by continuous states which, of course, assume an infinite number of values. In this case one can prove that the value function at step k , $\widehat{V}_k(s_t)$, can now be approximated by a parameterized functional form with parameter vector θ_k . It involves that the associated value function $\widehat{V}_k(s_t) = \widehat{V}_k(s_t; \theta_k)$ totally depends on θ_k , which varies step by step.

In order to estimate the optimal parameter vector, θ^* , which minimizes the “distance” between the unknown $V^\pi(s_t)$ and its estimate $\widehat{V}^\pi(s_t; \theta_k)$, in most learning approaches the minimization of the mean square error is used, that is:

$$\min_{\theta_k} \sum_s \left[V^\pi(s) - \widehat{V}^\pi(s; \theta_k) \right]^2.$$

The convergence of θ_k to θ^* is proven for approximators characterized by simple functional forms like linear ones, and it is also possible for particular complex functional forms like the ones involved like the so-called artificial neural networks([2]). Among the linear functional forms, in building our FTSs we use the following one:

$$\widehat{V}^\pi(s; \theta) = \sum_{i=1}^n \theta_i \phi_i(s_i) = \theta' \phi(s), \quad (7)$$

where n is the number of states and $\phi_i(\cdot)$ is a suitable transformation of the state. One can prove that, under mild assumptions, the update rules to use for estimating the state-action value function in the case of continuous states become:

$$d_k = r(s_t, a_t, s_{t+1}) + \gamma \max_a \widehat{Q}(s_{t+1}, a; \theta_k) - \widehat{Q}^\pi(s_t, a_t; \theta_k) \text{ and } \theta_{k+1} = \theta_k + \alpha d_k \nabla_{\theta_k} \widehat{Q}^\pi(s_t, a_t; \theta_k). \quad (8)$$

2.4 Kernel-based Reinforcement Learning

Another method for approximating $Q^\pi(s, a)$, alternative to the QLa and also usable in case of continuous states, is the nonparametric regression-based one known as *Kernel-based Reinforcement Learning* (KbRL) ([8], [9]). Given a kernel $K(\cdot)$ and defined $q_t = (s_t, a_t)$, the KbRL estimates $Q(s, a)$ as follows:

$$\hat{Q}_k(q_t) = \sum_{i=1}^{t-1} p_i(q_t) \hat{Q}_k(q_i),$$

where $p_i(q_t) = \frac{K(\frac{q_t - q_i}{h})}{\sum_{i=1}^{t-1} K(\frac{q_t - q_i}{h})}$. Note that the choice of the kernel $K(\cdot)$ is not crucial, whereas the choice of the bandwidth h has to be done carefully ([4]).

In this new reference frame, the approximation relationship for $\hat{Q}_{k+1}(q_t)$ is now given by:

$$\hat{Q}_{k+1}(q_i) = \hat{Q}_k(q_i) + p_i(q_t) [\hat{Q}_{k+1}(q_t) - \hat{Q}_k(q_i)], \quad (9)$$

wirh $i = 1, 2, \dots, t-1$.

This approach is interesting because constitutes a kind of minimization method without derivatives ([3]). Further, it is also usable in non-stationary contexts as the update relationship (9) is based on the current values of state-action pairs.

3 The Reinforcement Learning-based Financial Trading Systems

In this section we use the QLa and the KbRL for developing daily FTSs. First one has to identify the quantities which specify the states, the possible actions of the FTS, and the reward function.

With regards to the states, as at present we are mainly interested in testing the applicability of the considered RL methods to the development of FTSs, we simply use as states the last five percentage returns of the asset to trade, like in some of the cited literature. For this same reason we do not consider the transaction costs and other frictions. So, given the current price of the asset, p_t , the state of the system at the time step t is given by the vector

$$s_t = (e_{t-4}, e_{t-3}, e_{t-2}, e_{t-1}, e_t),$$

where $e_\tau = \frac{p_\tau - p_{\tau-1}}{p_{\tau-1}}$. Concerning the possible action of the FTS, we utilize the three following actions:

$$a_t = \begin{cases} -1 & \text{(sell signal)} \\ 0 & \text{(stay-out-from-the-market signal),} \\ 1 & \text{(buy signal)} \end{cases}$$

in which the stay-out-from-the-market implies the closing of whichever previously open position (if any). Note that in most of the specialized literature, like for instance in [7], only the sell signal and the buy one are considered. Finally, with reference to the reward function, following [7] we take into account the well known Sharpe ratio. In particular, as we wish an indicator that reacts enough quickly to the consequences of the actions of the FTS, we consider the Sharpe ratio calculated only

in the last $L = 5$ trading days (a stock market week) and in the last $L = 22$ trading days (a stock market month), that is:

$$r_t = \frac{E_L[g_{t-1}]}{\sqrt{\text{Var}_L[g_{t-1}]}} ,$$

where $E_L(\cdot)$ and $\text{Var}_L(\cdot)$ are, respectively, the sample mean operator and the sample variance one, and $g_t = a_{t-1}e_t$ is the gained/lost percentage return obtained at time step t as a consequence of the action taken by the FTS at time step $t - 1$.

Now let us pass to the two RL-based approaches we consider: The QLa and the KbRL. With respect the QLa, the kind of linear approximator of the state-action value function we choose is:

$$Q(s_t, a_t; \theta_k) = \theta_{k,0} + \sum_{n=1}^5 \theta_{k,n} \arctan(s_{t,n}) + \theta_{k,6} \arctan(a_t),$$

in which $\arctan(\cdot)$ plays the role of transformer of the state. Then, the ε -greedy function we follow is:

$$a_t = \begin{cases} \arg \max_u Q(s_t, a_t; \theta_k) & \text{with probability } 1 - \varepsilon, \\ u & \text{with probability } \varepsilon \end{cases}$$

in which $\varepsilon = 2.5\%$, 5.0% and $u \sim \mathcal{U}_d(-1, 1)$. Concerning the KbRL, we follow [4] and [8]. Note that, to the best of our knowledge, the use of the arctangent transformation in the QLa and of the KbRL for building FTSs is new.

Summarizing, we consider two different RL-based approaches (QLa and KbRL), two different values of ε (2.5% and 5.0%) and two different values of L (5 and 22), for a total of eight configurations.

We apply the above specified RL-based FTSs to two different time series of daily prices: An artificial one and a real one. With reference to the artificial time series, as in [7] we generate log price series as random walks with autoregressive trend processes. The used model is:

$$p_t = \exp \left\{ \frac{z_t}{\max z - \min z} \right\},$$

where $z_t = z_{t-1} + \beta_{t-1} + 3a_t$, in which $\beta_t = 0.9\beta_{t-1} + b_t$, $a_t \sim \mathcal{N}(0, 1)$, and $b_t \sim \mathcal{N}(0, 1)$. The length of the so-generated series is $T = 5000$. This artificial price series shows features which are often present in real financial price series. In particular, it is trending on short time scales and has a high level of noise. As far as the real time series regards, we utilize the closing prices of Banca Intesa and of Fiat (from the Italian stock market), from January 1, 1973 to September 21, 2006. The length of this series is $T = 5400$.

At this point we can present the results of the applications of the various configurations. In all the experimentations we set $\alpha = 0.8$ and $\gamma = 0.7$.

In figure 1 we graphically report the results of the application of the QLa-based FTS to the artificial price series, with $\varepsilon = 5\%$ and $L = 5$. In particular: The first panel shows the price series; The second panel shows the actions taken by the FTS at each time step, that is its investment strategy; The third panel shows the rewards, that is the Sharpe ratios, at each time step; The fourth panel shows the cumulative return one should obtain by investing the same monetary amount at

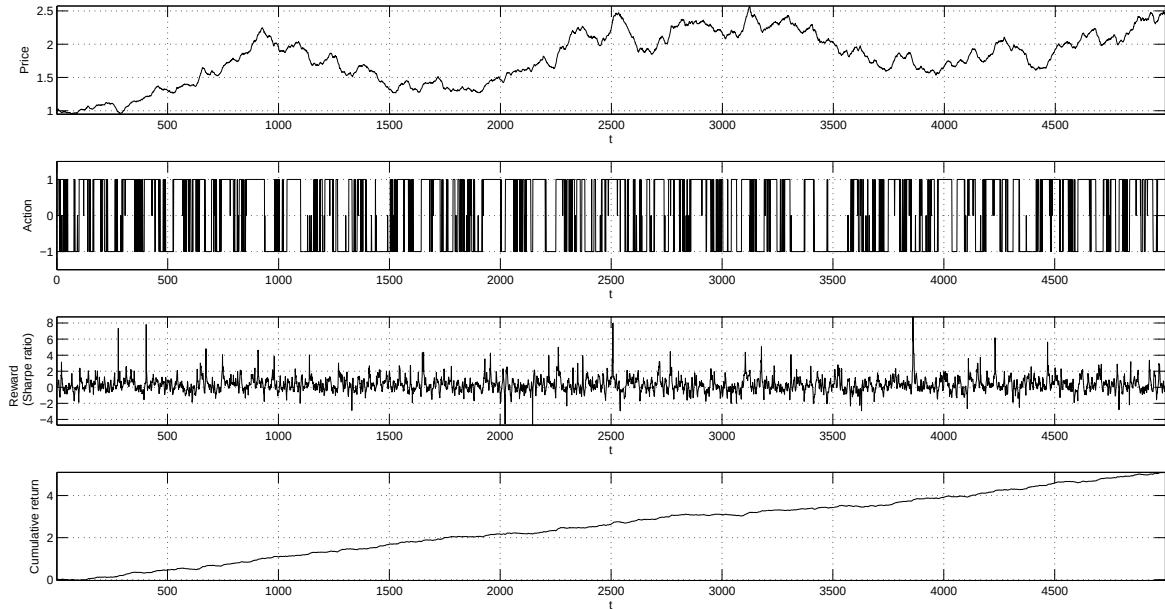


Figure 1: Results of the QLa-based FTS applied to the artificial price series, with $\varepsilon = 5\%$ and $L = 5$. Final cumulative returns: 510.86%.

each time step. At the end of the trading period, $t = T$, the cumulative return is 510.86%. In figure 2 we graphically report the results of the application of the QLa-based FTS to the real price series of Banca Intesa, with $\varepsilon = 5\%$ and $L = 5$. At the end of the trading period, $t = T$, the cumulative return is 271.42%.

It is very important to note that at the beginning of the trading period, $t = 0$, the vector of the parameters used in the linear approximator, θ_k , is randomly initialized. Because of it, by repeating some times this latter application we observe a certain variability in the final cumulative return: -158.52% , 174.91% , -26.38% This shows that the influence of the random initialization heavily spreads overall the trading period instead to soften as time step increases. In figure 3 we graphically report the results associated to the first of such repetition. To check the effects of this random initialization, we repeated 1000 times the application of each of the investigated configurations. When the application was to the artificial time series, the series has been taken the same in all the repetition. In table 1 we report some statistics concerning the final cumulative returns.

With reference to the KbRL, we obtain results similar to the one related to the QLa, although a bit less performing. As exemplification: 1) in figure 4 we graphically report the results of the application of the KbRL-based FTS to the real price series of Banca Intesa, with $\varepsilon = 5\%$ and $L = 22$ (at the end of the trading period, $t = T$, the cumulative return is 309.86%); 2) in figure 5 we graphically report the results of the application of the KbRL-based FTS to the real price series of Fiat, with $\varepsilon = 7.5\%$ and $L = 22$ (at the end of the trading period, $t = T$, the cumulative return is 135.47%). Note that for the KbRL there is not parameter vector to randomly initialize, but at the beginning of the trading period, $t = 0$, it is necessary to randomly initialize a suitable set of state-

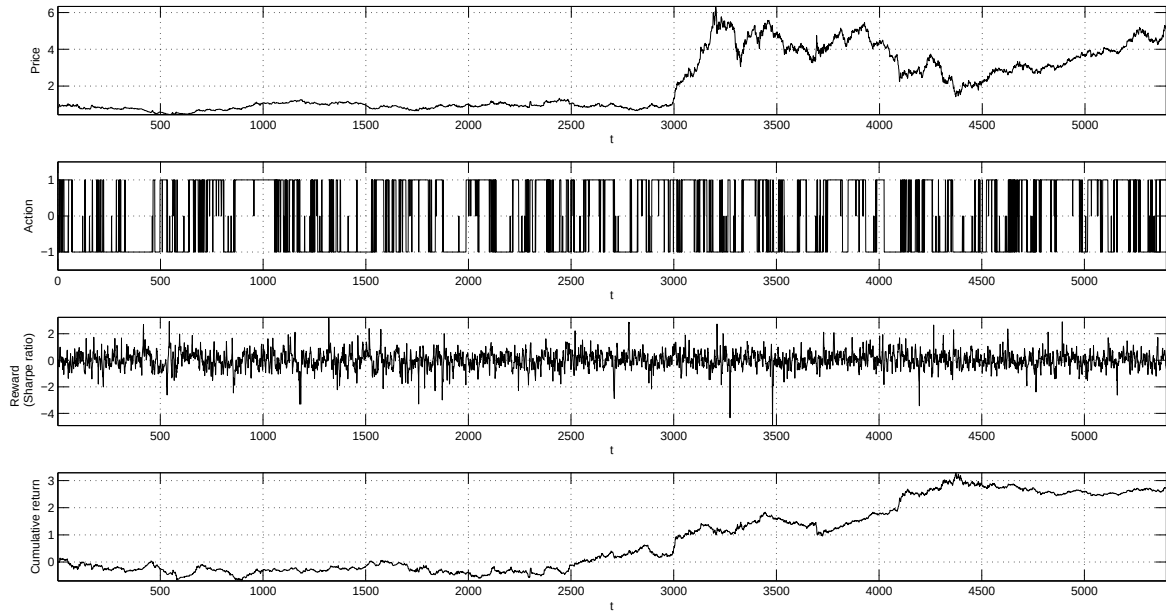


Figure 2: Results of the QLa-based FTS applied to the Banca Intesa price series, with $\varepsilon = 5\%$ and $L = 5$. Final cumulative returns: 271.42%.

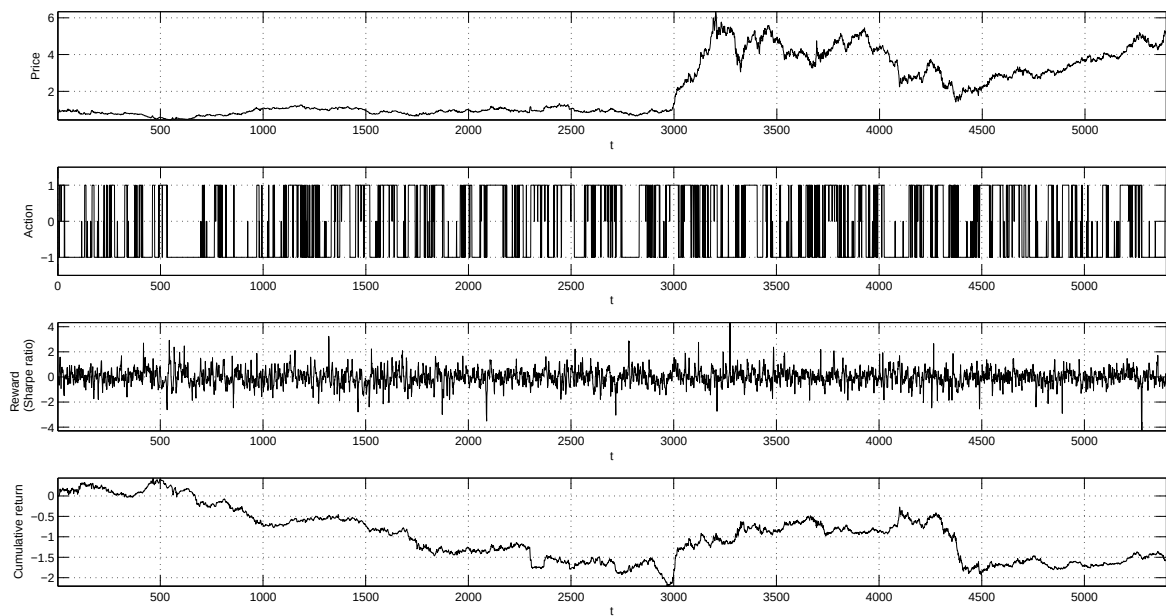


Figure 3: Results of the QLa-based FTS applied to the Banca Intesa price series, with $\varepsilon = 5\%$ and $L = 5$ (repetition). Final cumulative returns: -158.52% .

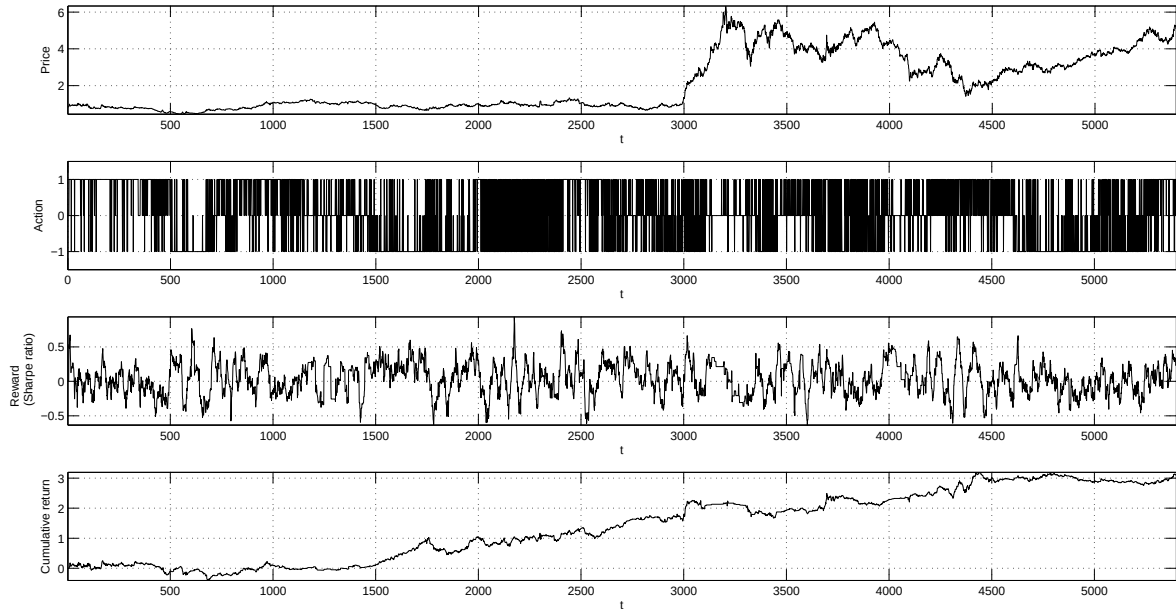


Figure 4: Results of the KbRL-based FTS applied to the Banca Intesa price series, with $\varepsilon = 5\%$ and $L = 22$. Final cumulative returns: 309.86%.

action pairs ([8]). So, also in this approach one puts the question of the variability of the results. As for the QLa, for the KbRL too we repeated 1000 times the application of each of the investigated configurations (see table 1 for some statistics about the final cumulative returns).

The main facts detectable from tables 1 are the following ones:

- Given the values of the means, most of the investigated configurations appears to be profitable, in fact the most of the means (88.89%) are positive. Further, the QLa approach generally seems more performing than the KbRL one;
- Given the values of the standard deviations, the results of all the considered configurations are characterized by a certain level of variability. In particular, such values emphasize that the question of the influence of the random initialization on the results is mainly true for the real financial time series;
- Given the results, it appears that the value of L has a significant impact on the performances of the FTSs. In particular, with reference to the artificial time series, all the checked FTS configurations are better performing when $L = 5$, whereas, with reference to both the real time series, the most of the checked FTS configurations (91.67%) are better performing when $L = 22$;
- Given the results, it appears that also the value of ε has a significant impact on the performances of the FTSs. In particular, although with reference to the artificial time series empir-

Approach	ε	L	Statistics	Artificial time series	Banca Intesa time series	Fiat time series
QLa	2.5%	5	μ σ Conf. interval	443.62% 37.83% [369.48%, 517.76%]	-3.77% 127.48% [-253.53%, 246.08%]	40.58% 139.35% [-232.55%, 313.70%]
QLa	2.5%	22	μ σ Conf. interval	307.03% 43.02% [222.71%, 391.35%]	70.85% 146.34% [-215.97%, 357.68%]	101.21% 144.27% [-181.55%, 383.98%]
QLa	5.0%	5	μ σ Conf. interval	472.02% 32.79% [407.76%, 536.29%]	40.00% 139.09% [-226.73%, 306.73%]	91.26% 137.30% [-177.85%, 360.37%]
QLa	5.0%	22	μ σ Conf. interval	337.68% 40.80% [257.71%, 417.64%]	92.92% 149.28% [-199.66%, 385.50%]	124.69% 144.89% [-159.29%, 408.68%]
QLa	7.5%	5	μ σ Conf. interval	467.48% 31.11% [406.50%, 528.46%]	49.15% 139.71% [-224.67%, 322.97%]	114.41% 142.42% [-164.73%, 393.56%]
QLa	7.5%	22	μ σ Conf. interval	339.04% 39.60% [261.43%, 416.66%]	99.66% 146.74% [-187.95%, 387.28%]	133.81% 147.53% [-155.34%, 422.96%]
KbRL	2.5%	5	μ σ Conf. interval	483.71% 60.10% [365.91%, 601.51%]	-26.16% 141.37% [-303.25%, 250.93%]	73.33% 144.00% [-208.90%, 355.56%]
KbRL	2.5%	22	μ σ Conf. interval	237.64% 63.55% [113.09%, 362.20%]	15.90% 159.92% [-297.54%, 329.34%]	75.37% 131.08% [-181.55%, 332.28%]
KbRL	5.0%	5	μ σ Conf. interval	435.42% 41.13% [354.81%, 516.02%]	-7.99% 131.26% [-265.26%, 249.28%]	77.32% 130.42% [-178.30%, 332.95%]
KbRL	5.0%	22	μ σ Conf. interval	216.61% 49.98% [118.64%, 314.58%]	13.20% 153.34% [-287.34%, 313.74%]	71.98% 136.04% [-194.66%, 338.61%]
KbRL	7.5%	5	μ σ Conf. interval	401.76% 39.80% [323.75%, 479.77%]	-0.63% 130.40% [-256.23%, 254.96%]	67.55% 128.00% [-183.33%, 318.42%]
KbRL	7.5%	22	μ σ Conf. interval	197.78% 42.55% [114.38%, 281.18%]	35.36% 143.51% [-245.91%, 316.63%]	74.82% 132.06% [-184.02%, 333.65%]

Table 1: Some statistics about the final cumulative returns.

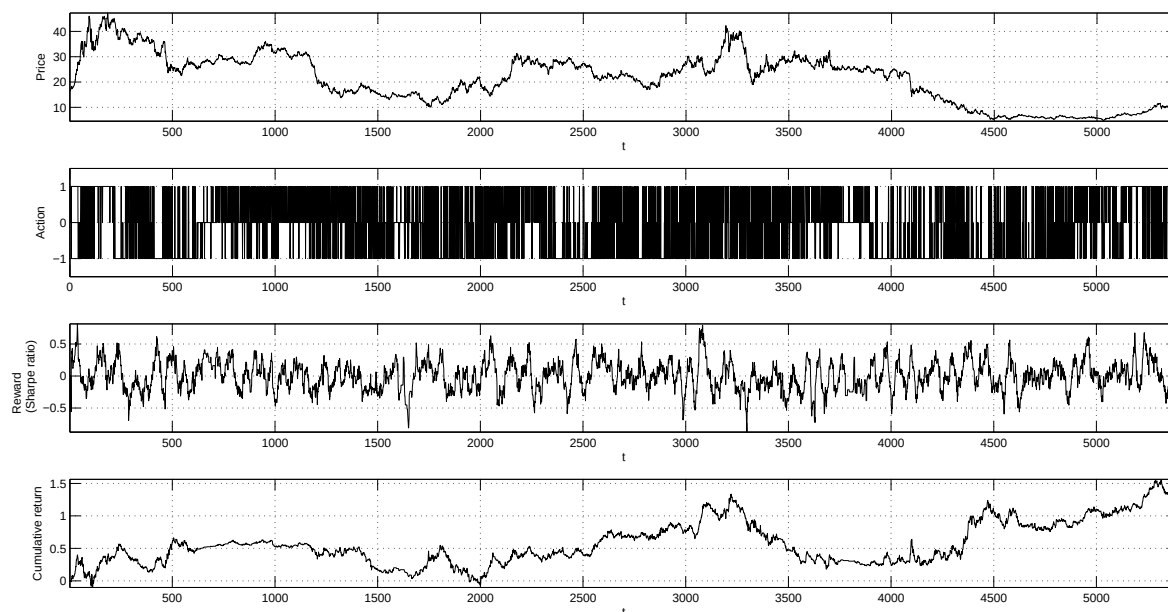


Figure 5: Results of the KbRL-based FTS applied to the Fiat price series, with $\varepsilon = 7.5\%$ and $L = 22$. Final cumulative returns: 135.47%.

ical regularities does not appear, with reference to both the real time series, the most of the checked FTS configurations (75.00%) are better performing when $\varepsilon = 7.5\%$.

4 Some concluding remarks

In this paper we have developed and applied some original automatic FTSs based on differently configured RL algorithms. Here we have presented the results coming out from the current phase of our research on this topic. Of course, many questions have again to be explored. In particular:

- The choice of the last five percentage returns as states is a naive choice. Now we are beginning to work to specify some new indicators to use as states (in the first experimentations they have provided interesting results);
- As known, the Sharpe ratio as performance measure suffers several limits. Currently, as reward function we are considering alternative and more realistic performance measures;
- The management of the learning rate, α , we have used here is appropriate for stationary systems. But generally financial markets are non-stationary. Because of that, we are beginning to work to develop methods for the dynamic management of the learning rate in non-stationary contexts;

- In order to deepen the valuation about the capabilities of our FTSs, we wish to apply them to more and more financial price series coming from different markets;
- Finally, when all the previous questions will be explored, transaction costs and other frictions will be considered.

Acknowledgements

The authors wish to thank the Department of Economics of the Ca' Foscari University of Venice for the support received within the research project *Machine Learning adattativo per la gestione dinamica di portafogli finanziari* [Adaptive Machine Learning for the dynamic management of financial portfolios]).

References

- [1] Barto A.G., Sutton R.S. (1998), *Reinforcement Learning: An Introduction*. Adaptive Computation and Machine Learning. The MIT Press.
- [2] Bertsekas D.P., Tsitsiklis J.N. (1996), *Neuro-Dynamic Programming*. Athena Scientific.
- [3] Brent R.P. (1973), *Algorithms for Minimization without Derivatives*. Prentice-Hall.
- [4] Bosq D. (1996), *Nonparametric Statistics for Stochastic Processes. Estimation and Prediction* [Lecture Notes in Statistics, vol. 110]. Springer-Verlag.
- [5] Gold C. (2003), FX trading via recurrent Reinforcement Learning, *Proceedings of the IEEE International Conference on Computational Intelligence in Financial Engineering*, 363-370.
- [6] Moody J., Wu L., Liao Y., Saffel M. (1998), Performance functions and Reinforcement Learning for trading systems and portfolios, *Journal of Forecasting*, 17, 441-470.
- [7] Moody J., Saffel M. (2001), Learning to trade via Direct Reinforcement, *IEEE Transactions on Neural Network*, 12, 875-889.
- [8] Ormonet D. (2002), Kernel-Based Reinforcement Learning, *Machine Learning*, 49, 161-178.
- [9] Smart W.D., Kaelbling L.P. (2000), Practical Reinforcement Learning in continuous spaces, *Proceedings of the 17th International Conference on Machine Learning*, 903-910.